

ARCHITECTURE MONOLITHIQUE MODERNE & SSR NATIVE

CAHIER DES CHARGES & ROADMAP DE DÉVELOPPEMENT MONOLITHE

Plateforme BIOLINKO 100% Monolithique : Laravel 13 + Inertia.js v2 + React 19 + Tailwind CSS

DOCUMENT DE CONCEPTION ET PLAN DE DÉVELOPPEMENT V4

Projet : BIOLINKO (biolinko.app)

Architecture : Monolithe Réactif (Zero API REST Interne / Zero Next.js)

Auteur : Kouongme Mbouom Franck Dimitri

Poste : Full Stack Lead Architect & CMO

Date : Juillet 2026

Statut : Architecture Définitive Validée

TABLE DES MATIÈRES

1. Clarification Archi : Le Modèle Monolithe Inertia.js	3
2. Schema de la Base de Données Monolithique (PostgreSQL)	3
3. Plan de Développement Modulaire (Roadmap Code Monolithe)	4
Module 1 : Socle Monolithe Laravel 13 + Inertia React + Auth Session	
Module 2 : Dashboard Vendeur React & SSR SmartLinks (Routing Laravel)	
Module 3 : Fast Checkout Client Native Monolithe & Calculs Financials	
Module 4 : Intégration Passerelle HR-PAY (Push USSD & Webhook Inbound)	
Module 5 : Wallet Vendeur, Demand System & Retraits MoMo (2%)	
Module 6 : Jobs Asynchrones Redis & Relances WhatsApp	
4. Planning d'Exécution Hebdomadaire (Sprint 4 Semaines)	7

1. CLARIFICATION ARCHI : LE MODÈLE MONOLITHE INERTIA.JS

Afin de maximiser la vitesse de développement, la simplicité de déploiement et la cohérence de la base de code, le projet **BIOLINKO** est conçu comme un **Monolithe Moderne** unique :

- Suppression Complète de l'Architecture Découplée (Zero Next.js, Zero API REST Interne) : Il n'y a pas deux projets séparés ni de gestion complexe de tokens OAuth/Sanctum.
- Laravel 13 + Inertia.js v2 + React 19 : Laravel gère directement le routage, la sécurité des sessions, la base de données PostgreSQL, les jobs Redis et injecte directement les données props dans les composants React monolithiques via `Inertia::render('Store/Show', $props)`.
- Inertia SSR (Server-Side Rendering) : La partie publique (vitrines vendeurs et SmartLinks) bénéficie du moteur SSR natif d'Inertia pour afficher les pages sous 800ms sans API tierce.

2. SCHÉMA DE LA BASE DE DONNÉES MONOLITHIQUE (POSTGRESQL)

Les tables relationnelles gérées directement par Eloquent ORM au sein du monolithe :

TABLE	CHAMPS STRUCTURANTS CLÉS	RÔLE ET CLÉS ÉTRANGÈRES
users	id, name, email, phone_whatsapp, password, role	Comptes vendeurs et administrateurs. Authentification web classique par session.
stores	id, user_id, name, slug, logo_url, banner_url, theme_color, plan_type	Paramètres de la vitrine. slug unique (ex: luxestyle). Libre affichage des contacts.
products	id, store_id, title, slug, price_vendor, stock, is_active	Catalogue d'articles. price_vendor (ex: 5000 FCFA).
product_variants	id, product_id, size, color, stock_quantity	Variantes de tailles/couleurs.
orders	id, store_id, product_id, customer_name, customer_phone, city, price_vendor, saas_margin, api_fee, total_client, status, payment_status	Commandes transactionnelles. payment_status (pending, paid, abandoned).
wallets	id, store_id, balance_available, balance_pending	Solde du vendeur crédité à 100% du prix du produit.
withdrawals	id, wallet_id, amount_requested, fee_api, fee_saas, net_transferred, phone_momo, status	Demandes de retraits Mobile Money avec commission 2%.

3. PLAN DE DÉVELOPPEMENT MODULAIRE (ROADMAP CODE MONOLITHE)

MODULE 1 : Socle Monolithe Laravel 13 + Inertia React + Auth Session

Objectif : Mettre en place le monolithe Laravel avec Inertia, Tailwind et le système d'authentification par session Web.

- **Étape 1.1 :** Installer Laravel 13, Inertia.js v2, React 19 et Tailwind CSS (`composer require inertiajs/inertia-laravel`).
- **Étape 1.2 :** Exécuter les migrations PostgreSQL pour toutes les tables (`users`, `stores`, `products`, etc.).
- **Étape 1.3 :** Installer Laravel Breeze (stack Inertia React) pour l'authentification par session Web classique.
- **Étape 1.4 :** Écouter l'événement Inscription Vendeur pour créer automatiquement le `store` (`biolinko.app/[slug]`) et le `wallet`.

MODULE 2 : Dashboard Vendeur React & SSR SmartLinks (Routing Laravel)

Objectif : Construire les vues React d'administration boutique et la résolution des URLs canoniques.

- **Étape 2.1 :** Contrôleur Monolithe `StoreController` et Vues React pour la personnalisation du Profil (Logo, Bannière, Thème Sombre/Purple, Réseaux Sociaux, Contacts libres).
- **Étape 2.2 :** CRUD Produit avec `useForm` d'Inertia (Gestion du titre, prix \$P_v\$, upload images, variantes et stocks).
- **Étape 2.3 :** Routage Web Laravel direct pour les SmartLinks :
 - Route Boutique : `Route::get('/{store:slug}', [PublicStoreController::class, 'show']);`
 - Route Produit Direct : `Route::get('/{store:slug}/p/{product:slug}', [PublicProductController::class, 'show']);`
- **Étape 2.4 :** Contrôle des quotas des 4 abonnements (Starter 0F, Premium 7k, Pro 16k, Growth 30k) via Middleware Laravel.

MODULE 3 : Fast Checkout Client Native Monolithe & Calculs Financiers

Objectif : Rendre la page publique réactive avec le formulaire Fast Checkout et la formule de calcul financière.

- **Étape 3.1 :** Configurer Inertia SSR (`php artisan inertia:start-ssr`) pour un rendu HTML ultra-rapide (< 800ms) sur navigateurs mobiles In-App Webviews.
- **Étape 3.2 :** Formulaire de Checkout React 1-Click (Nom, Quartier, Téléphone Mobile Money).
- **Étape 3.3 :** Intégrer la formule de calcul exacte du prix dans le contrôleur Laravel :

```
$P_b = P_v imes 1.02$ (Base Prix Plateforme)
```

```
$T_c = Math.ceil(P_b / 0.98)$ (Total Payé par le Client Final)
```

```
$F_{api} = T_c - P_b$ (Frais réels 2% HR-PAY)
```

MODULE 4 : Intégration Passerelle HR-PAY (Push USSD & Webhook Inbound)

Objectif : Déclencher le paiement en ligne et traiter la confirmation en tâche de fond.

- **Étape 4.1 :** Action de soumission de commande : création de l'enregistrement `Order` en statut `pending` et appel du service PHP HR-PAY pour envoyer l'invite USSD sur le téléphone de l'acheteur.
- **Étape 4.2 :** Route Webhook externe isolée (`Route::post('/webhooks/hrpay', ...)` sans vérification CSRF) :
 - Valider la signature du webhook HR-PAY.
 - Passer `order.payment_status = 'PAID'`.
 - Créditer `wallet.balance_available += order.price_vendor` (ex: +5 000 FCFA nets pour le vendeur).
 - Déclencher la notification in-app et WhatsApp pour le vendeur.

MODULE 5 : Wallet Vendeur, Demand System & Retraits MoMo (2%)

Objectif : Gérer la trésorerie du vendeur et l'exécution des retraits Mobile Money.

- **Étape 5.1 :** Page React Inertia `Wallet/Index` affichant le solde disponible et l'historique des ventes.
- **Étape 5.2 :** Soumission du formulaire de retrait en Inertia (`useForm`) vers `WithdrawalController@store`.
- **Étape 5.3 :** Calcul monolithique de la commission de retrait de 2% :
 - `fee_api = amount * 0.01` (1% pour HR-PAY)
 - `fee_saas = amount * 0.01` (1% de marge nette BIOLINKO)
 - `net_transferred = amount * 0.98`
- **Étape 5.4 :** Appel du service de virement sortant HR-PAY et mise à jour du Wallet.

MODULE 6 : Jobs Asynchrones Redis & Relances WhatsApp

Objectif : Automatiser la relance des paniers abandonnés via les Laravel Queues nativement intégrées au monolithe.

- **Étape 6.1 :** Commande planifiée Laravel Scheduler (`Schedule::command('orders:check-abandoned')`) identifiant les paniers initiés non payés sous 30 min.
- **Étape 6.2 :** Dispatch du Job Redis `SendWhatsAppReminderJob` pour envoyer le rappel automatique WhatsApp aux clients des vendeurs Pro/Growth.
- **Étape 6.3 :** Bouton "Direct WhatsApp" dans le Dashboard Vendeur React générant directement le lien `wa.me/[phone]?text=[encoded_text]`.

4. PLANNING D'EXÉCUTION HEBDOMADAIRE (SPRINT 4 SEMAINES MONOLITHE)

Grâce à la simplification du monolithe (une seule application à coder et à déployer) :

SEMAINE	MODULES MONOLITHE	LIVRABLES CLÉS DE LA SEMAINE
Semaine 1	Module 1 & Module 2	Monolithe Laravel 13 + Inertia React initialisé, BDD migrée, Auth Session, Dashboard Profil Vendeur & CRUD Catalogue avec SmartLinks.
Semaine 2	Module 3 & Module 4	Vitrines publiques Inertia SSR (< 800ms), Fast Checkout réactif, Formules de calcul 2%, Service HR-PAY et Controller Webhook.
Semaine 3	Module 5 & Module 6	Page Wallet React, Moteur de Retrait MoMo (commission 2%), Laravel Scheduler & Queues Redis pour relances WhatsApp.
Semaine 4	Polissage UI & Deploy	Styling Tailwind CSS (Thème Sombre / Purple), tests de bout en bout (E2E), Landing Page marketing et déploiement sur un VPS unique.

Avantage Majeur du Monolithe : Un seul dépôt Git, un seul serveur d'hébergement, zero problème de CORS ou de jetons API expirés. C'est l'architecture idéale pour développer vite, proprement et être prêt le mois prochain !